DÍA MES AÑO NOTA /

Summaries

Summary 1: Polymorphism

Polymorphism comes from the greek term meaning "having many forms" and stands as one of the three foundational pillars of Object oriented programming alongside Encapsulation and Inheritance. Its primary purpose is to allow client code to treat related objects as if they were of the same common type. Although the client code interacts with these objects through a unified abstraction, each specific object executes its correct, specialized behavior at runtime

Without polymorphism, code bases often rely heavily on type checks using conditional statements like instanceof followed by manual object casting. This approach leads to a maintenance nightmare because adding any new object type requires updating every conditional block across the codebase. Furthermore, it makes the code verbose, error prone, and poorly designed, as it forces the system to inspect an object's identity before telling it how to act rather than simply delegating the task directly.

Implementing polymorphism requires establishing a common superclass or interface with an abstract method that enforces a contract across all concrete subclasses. Subclasses then override this method using @Override to supply their specific logic. Consequently, the client code can depend entirely on the abstraction and invoke the method directly, producing cleaner, shorter code that is easily extensible. The primary trade off is that precise runtime behavior cannot be determined until execution, trading static certainty for architectural flexibility.

In Java collections, declaring a list with a superclass type allows it to hold any valid subclass instance, whereas a list declared with a specific subclass cannot hold superclass instances. Additionally, Java distinguishes between method overriding and method overloading. Overriding redefines and inherited method in a subclass to provide customized behavior, whereas overloading defines multiple methods within the same class using identical names but different parameter signatures

Summary 2: Abstraction

Abstraction as a process involves sifting through an object's attributes and behaviors to isolate only the details relevant to the specific context of the system while ignoring everything else. As a noun, an abstraction represents the resulting class or interface definition that captures these essential features. For example, a chicken class in a farm management system includes attributes like ID, breed, and birth date while ommitting irrelevant details such as physical shape, exact weight, or personal philosophy

Nivels of abstraction determine how generalized or detailed a class is within a system. A base class like ChessPiece is more abstract than a Pawn subclass because it contains fewer specific details and higher level general concepts. Effective software design relies on identifying shared similarities

DÍA MES AÑO NOTA /

across system components, moving those common features into an abstract layer, and delegating specific differences to concrete subclasses. Because abstract layers leave out implementation specifics, they cannot stand on their own and require concrete implementations to functions

Abstraction works hand in hand with encapsulation and detail hiding to maintain modular software design. Public properties and methods define the accessible interface exposed to external code, while private variables and internal implementation details remain hidden. This localization of design decisions of design decisions ensures that internal modifications to a component do not ripple through or break the surrounding codebase

Summary 3: Abstract classes and Interfaces.
Java provides specific language constructs to enforce abstraction, primarily using the abstract and interface keywords. Declaring a class with abstract prevents direct instantiation using the new keyword. An abstract method contains no body implementation and forces concrete subclasses to implement its logic. Interfaces define pure behavioral contracts that classes implement using the implements keyword, while subclasses inherit from parent classes using extends.

While Java strictly forbids instantiating an interface or abstract class directly, it fully supports polymorphic variable assignment. Declaring a variable under an interface or abstract supertype and instantiating it with a concrete implementing class such as assigning and ArrayList to a collection reference or a chicken to an IFarmAnimal reference is standar object oriented practice.

Abstract classes and interfaces serve distinct roles in Java design. An abstract class is extended by subclasses to share common state and base code while allowing partial implementation. In contrast, an interface defines a contract of required behaviors that any implementing class must satisfy regardless of its position in the class hierarchy. Both structures prevent direct instantiation, but interfaces focus on defining pure functionality while abstract classes provide a shared foundation for related objects.